

키움 트레이딩뷰 웹훅 서버 설치 가이드

이 가이드는 트레이딩 뷰 차트 웹훅 기능을 통해 자동으로 키움증권 주문을 실행하는 로컬 서버 설치 방법을 설명하는 키움증권 공식 가이드입니다. 사용자는 키움 REST API 사용신청을 완료하여 IP등록 및 APP키를 발급받은 것을 전제로 작성하였습니다. 사용 신청이 안된 사용자는 키움 REST API 홈페이지를 통해 IP등록 및 APP키 발급을 선행 해주시기 바랍니다.

- 키움 REST API 홈페이지 : <https://openapi.kiwoom.com/main/home>

※ 웹훅이란 특정 앱에서 이벤트가 발생했을 때(예: '결제완료', '로그인'), 해당 정보를 실시간으로 다른 서버나 프로그램에 자동으로 보내주는 기능을 말함

1. 기본 프로그램 설치

(1) node.js 다운로드 후 설치

- 공식 사이트 : <https://nodejs.org/ko/download>

(2) git 다운로드 후 설치

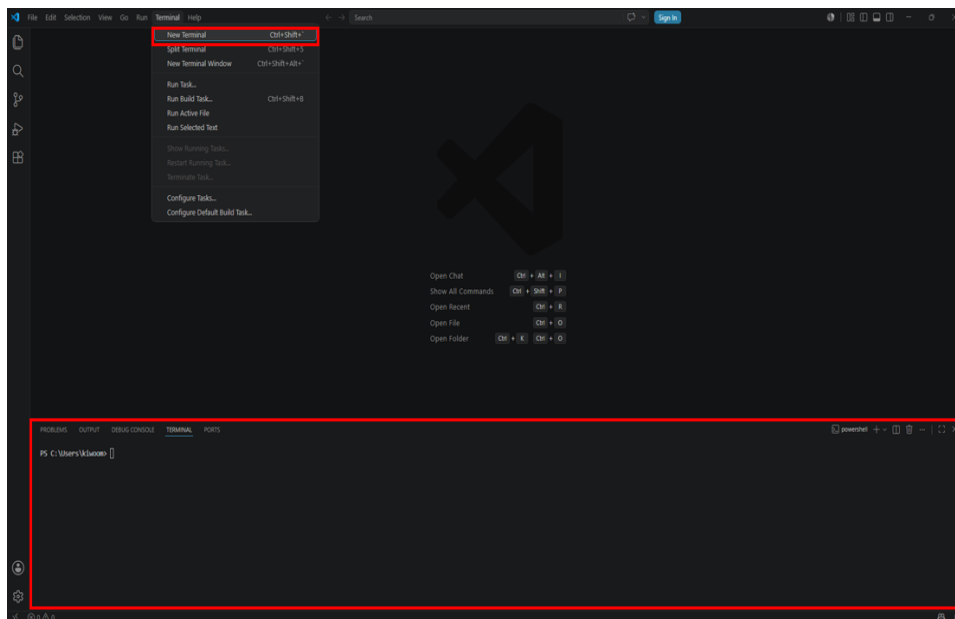
- 공식 사이트 : <https://git-scm.com/install/windows>

(3) vscode 다운로드 후 설치

- 공식 사이트 : https://code.visualstudio.com/download?_exp_download=fb315fc982

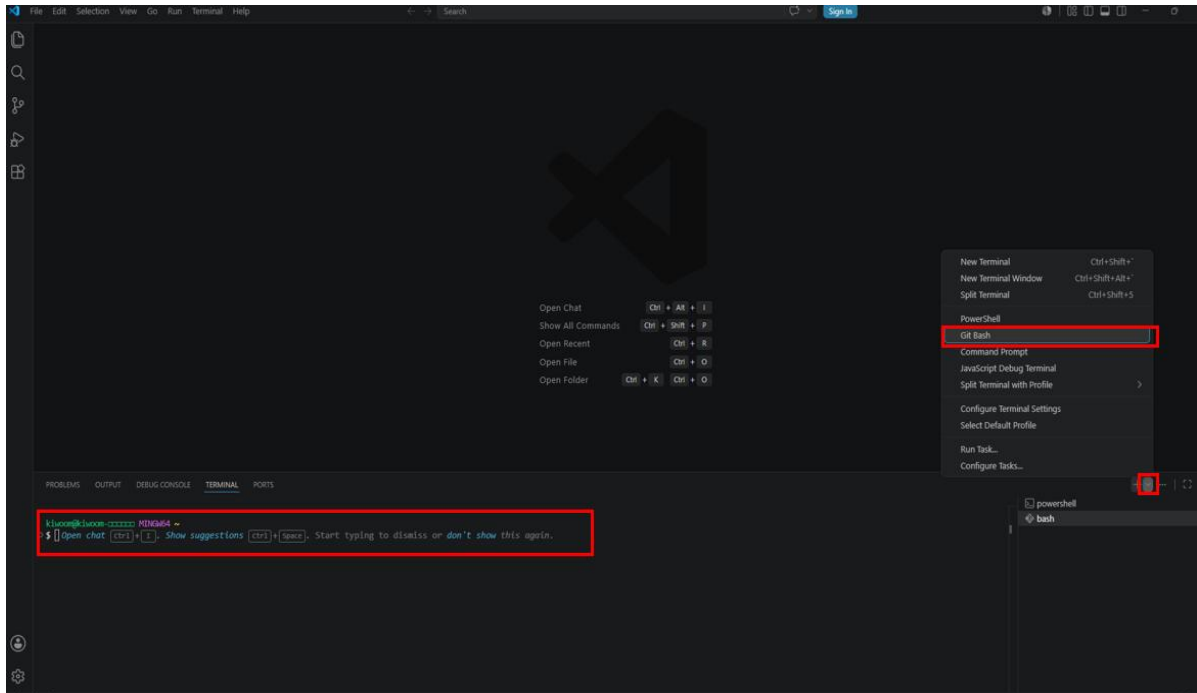
(4) 설치 확인

① vscode 실행 -> Terminal -> New Terminal 메뉴 선택하여 하단 터미널 창 오픈



② 하단 터미널 창 우측  버튼 클릭 후 Git Bash 선택하여 터미널 오픈

※ 앞으로의 가이드는 Git Bash기준이며, 글 머리기호 #이 실행해야 할 명령어입니다.



③ 아래 명령어 실행하여 설치 여부를 확인, 하단 화면과 같이 나온다면 설치 완료(버전의 숫자는 다를 수 있음)

```
# node -v
```

```
# npm -v
```

```
# git --version
```

```
kiwoom@kiwoom- MINGW64 ~
$ node -v
v24.18.0

kiwoom@kiwoom- MINGW64 ~
$ npm -v
11.16.0

kiwoom@kiwoom- MINGW64 ~
$ git --version
git version 2.55.0.windows.2
```

2. 키움 트레이딩뷰 웹훅 서버 구동

(1) 깃 허브 샘플 소스 클론

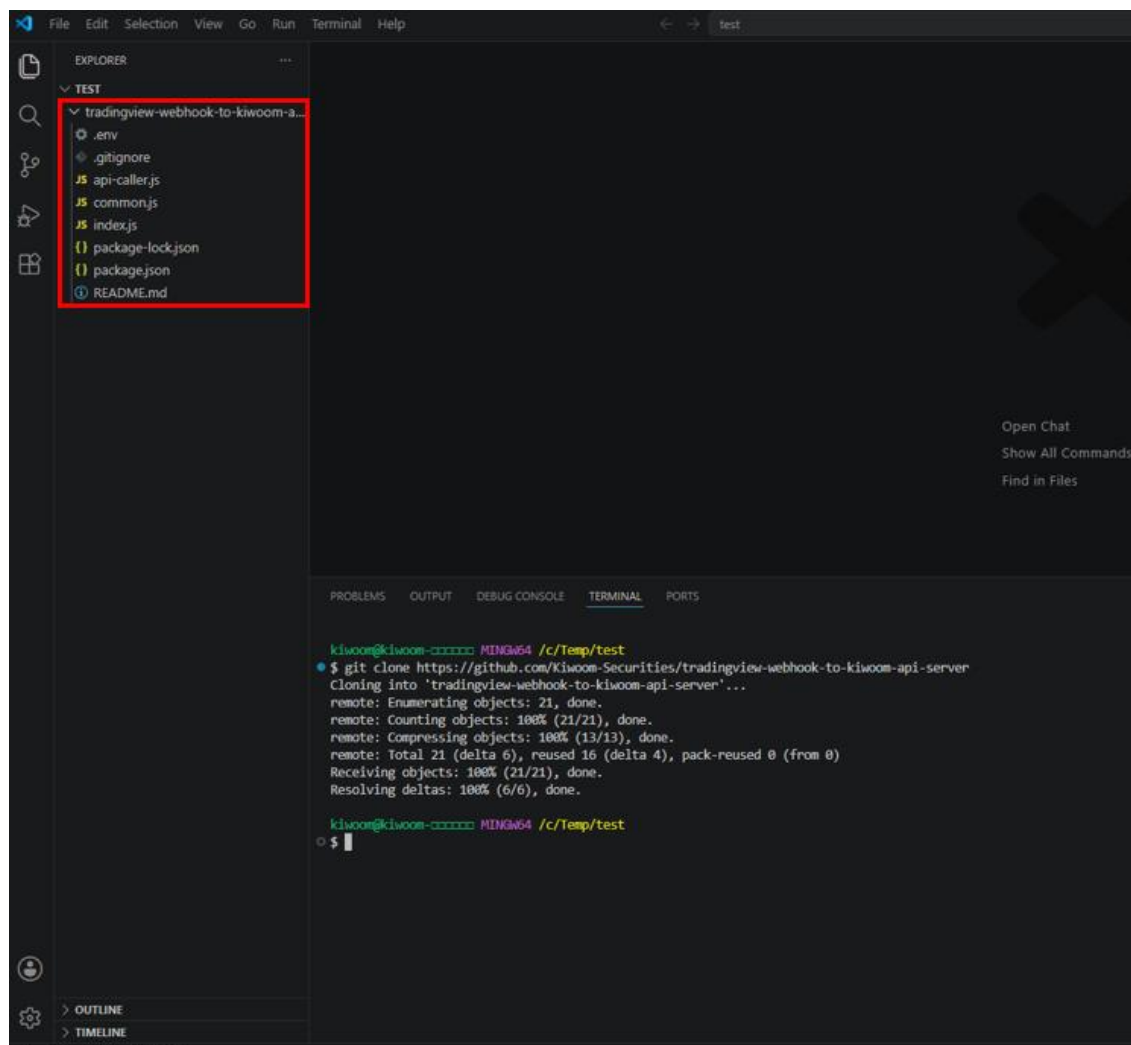
```
# git clone https://github.com/Kiwoom-Securities/tradingview-webhook-to-kiwoom-api-
```

server

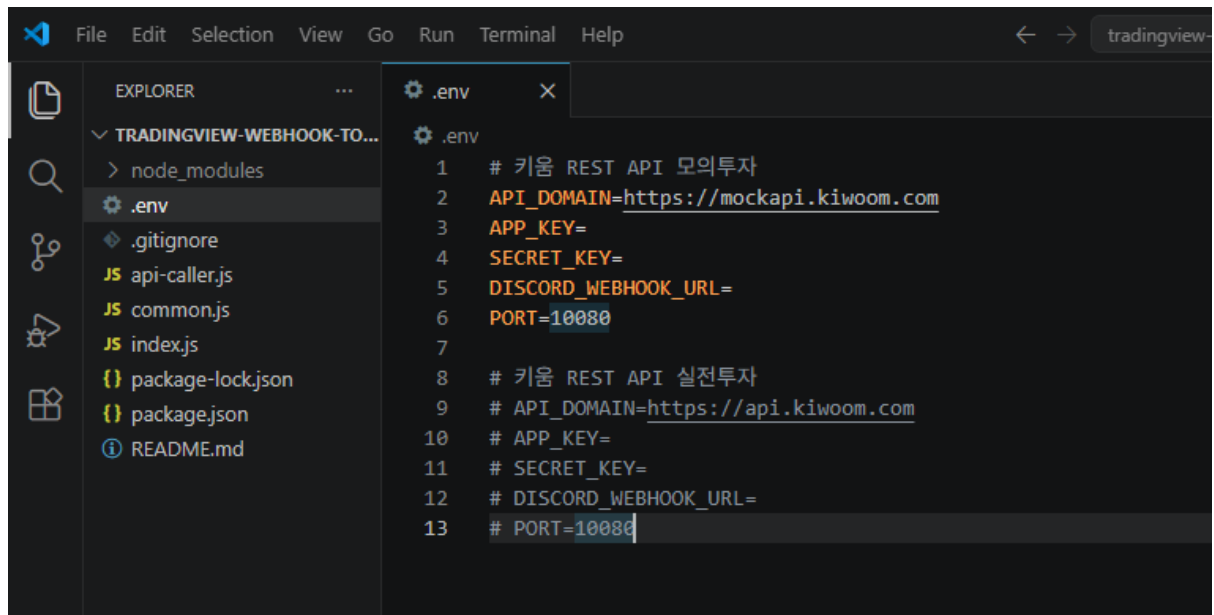
```
kiwoom@kiwoom-xxxxxx MINGW64 /c/Temp/test
$ git clone https://github.com/Kiwoom-Securities/tradingview-webhook-to-kiwoom-api-server
Cloning into 'tradingview-webhook-to-kiwoom-api-server' ...
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 21 (delta 6), reused 16 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (21/21), done.
Resolving deltas: 100% (6/6), done.

kiwoom@kiwoom-xxxxxx MINGW64 /c/Temp/test
$
```

(2) 샘플 소스 파일 생성되었는지 확인



(3) 모의투자/실전투자 선택하여 .env 환경 설정 입력



- API_DOMAIN : 키움 REST API 도메인, 수정 필요 없음
- API_KEY : 키움 REST API 홈페이지에서 발급 받은 내 계좌의 App Key를 입력
- SECRET_KEY : 키움 REST API 홈페이지에서 발급 받은 내 계좌의 App Secret를 입력
- DISCORD_WEBHOOK_URL : 매매 주문 응답 결과를 디스코드 메시지로 받고 싶은 경우에만 입력(필수 아님)
- PORT : 키움 트레이딩뷰 웹훅 서버의 포트, 다른 포트 사용시 수정

(4) 키움 트레이딩뷰 웹훅 서버 구동 방법

① 샘플 소스 경로로 이동 및 npm install 실행하여 관련 패키지 다운로드

```
# cd tradingview-webhook-to-kiwoom-api-server/
```

```
# npm install
```

```
# npm install nodemon -g
```

```
# npm install pm2 -g
```

```
kiwoom@kiwoom-000000 MINGW64 /c/Temp/test
$ cd tradingview-webhook-to-kiwoom-api-server/

kiwoom@kiwoom-000000 MINGW64 /c/Temp/test/tradingview-webhook-to-kiwoom-api-server (main)
$ npm install

up to date, audited 69 packages in 815ms

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

kiwoom@kiwoom-000000 MINGW64 /c/Temp/test/tradingview-webhook-to-kiwoom-api-server (main)
$ npm install nodemon -g

changed 28 packages in 810ms

5 packages are looking for funding
  run `npm fund` for details

kiwoom@kiwoom-000000 MINGW64 /c/Temp/test/tradingview-webhook-to-kiwoom-api-server (main)
$ npm install pm2 -g

changed 77 packages in 3s

8 packages are looking for funding
  run `npm fund` for details
```

② 아래 2가지 방법 중 한가지 선택하여 실행(빨간박스의 로그가 출력되면 정상)

방법1) # npm run local

```
kiwoom@kiwoom-000000 MINGW64 /c/git/tradingview-webhook-to-kiwoom-api-server (main)
$ npm run local

> tradingview-webhook-to-kiwoom-api-server@1.0.0 local
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injected env (5) from .env // tip: ✎ suppress logs { quiet: true }
2026-07-16 09:38:01 => start! tradingview-webhook-to-kiwoom-api server on port 10080
```

방법2)

pm2 start index.js (백그라운드 실행)

```
kiwoom@kiwoom-000000 MINGW64 /c/git/tradingview-webhook-to-kiwoom-api-server (main)
$ pm2 start index.js
[PM2] Starting C:\git\tradingview-webhook-to-kiwoom-api-server\index.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	namespace	version	mode	pid	uptime	↓	status	cpu	mem	user	watching
0	index	default	1.0.0	fork	9896	0s	0	online	0%	77.0mb	kiwoom	disabled

```
host metrics | cpu: 3.3% | ram usage: 67.8% | disk: ↓ 0mb/s ↑ 0mb/s |
```

pm2 logs (백그라운드 실행 로그 확인)

```
kiwoom@kiwoom-000000 MINGW64 /c/git/tradingview-webhook-to-kiwoom-api-server (main)
$ pm2 logs
[TAILING] Tailing last 15 lines for [all] processes (change the value with --lines option)
C:\Users\kiwoom\.pm2\pm2.log last 15 lines:
PM2      | 2026-07-16T09:40:33: PM2 log: Stopping app:index id:0
PM2      | 2026-07-16T09:40:35: PM2 log: App [index:0] exited with code [1] via signal [SIGINT]
PM2      | 2026-07-16T09:40:35: PM2 log: pid=12428 msg=process tree killed (1 pids)
PM2      | 2026-07-16T09:40:40: PM2 log: App [index:0] starting in -fork mode-
PM2      | 2026-07-16T09:40:40: PM2 log: App [index:0] online

C:\Users\kiwoom\.pm2\logs\index-error.log last 15 lines:
C:\Users\kiwoom\.pm2\logs\index-out.log last 15 lines:
0|index  | ◇ injected env (5) from .env // tip: ✎ secrets for agents [www.dotenvx.com]
0|index  | 2026-07-16 09:40:40 => start! tradingview-webhook-to-kiwoom-api server on port 10080
```

pm2 stop index | pm2 delete index(백그라운드 실행 중지 및 프로세스 삭제)

```
kiwoom@kiwoom-000000 MINGW64 /c/git/tradingview-webhook-to-kiwoom-api-server (main)
$ pm2 stop index
[PM2] Applying action stopProcessId on app [index](ids: [ 0 ])
[PM2] [index](0) ✓
```

id	name	namespace	version	mode	pid	uptime	↓	status	cpu	mem	user	watching
0	index	default	1.0.0	fork	0	0	0	stopped	0%	0b	kiwoom	disabled

```
host metrics | cpu: 3.7% | ram usage: 67.9% | disk: ↓ 0mb/s ↑ 0mb/s |

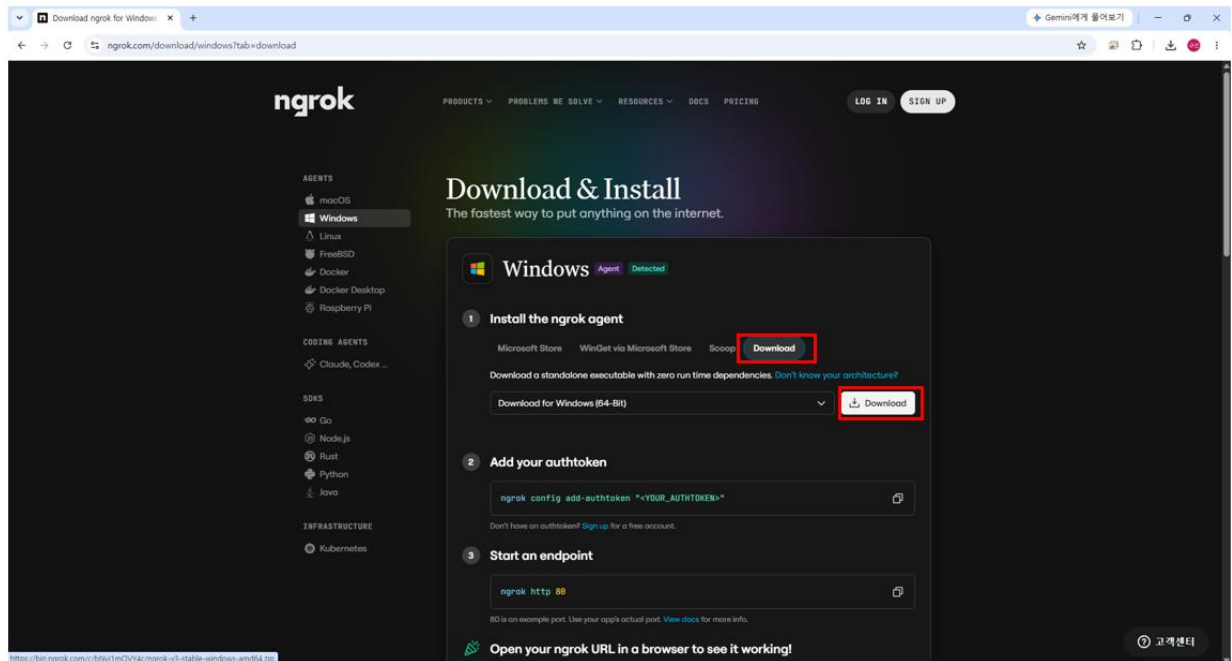
kiwoom@kiwoom-000000 MINGW64 /c/git/tradingview-webhook-to-kiwoom-api-server (main)
$ pm2 delete index
[PM2] Applying action deleteProcessId on app [index](ids: [ 0 ])
[PM2] [index](0) ✓
```

id	name	namespace	version	mode	pid	uptime	↓	status	cpu	mem	user	watching
----	------	-----------	---------	------	-----	--------	---	--------	-----	-----	------	----------

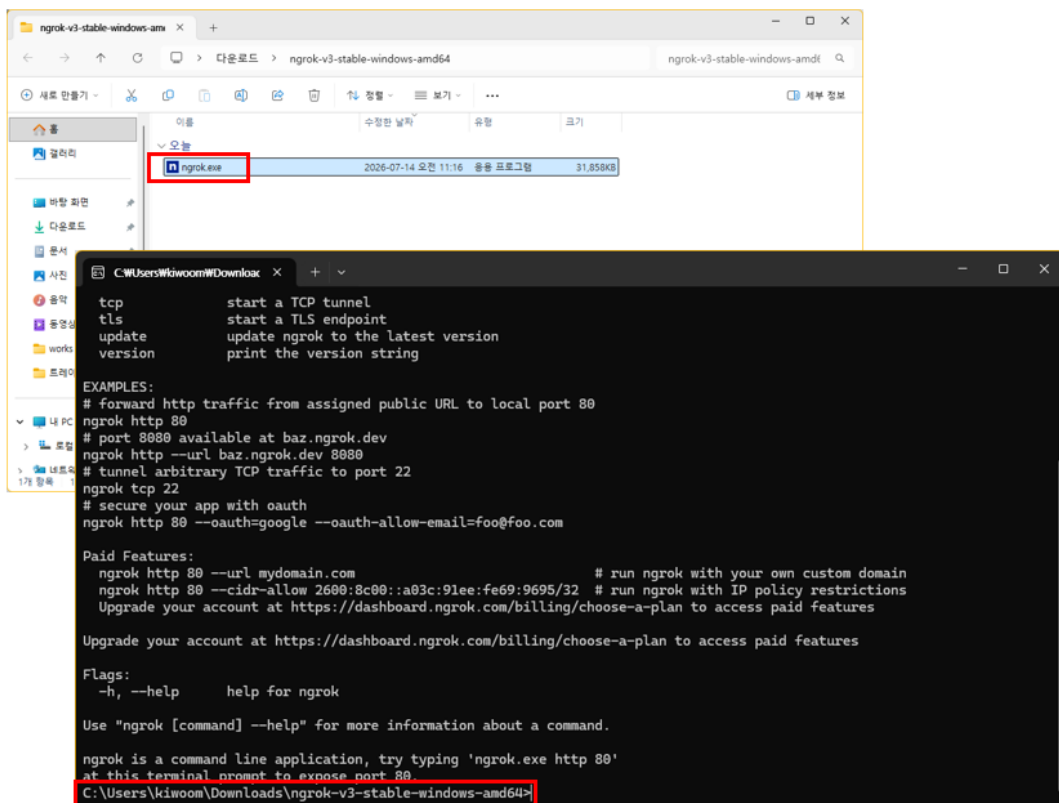
```
host metrics | cpu: 3.7% | ram usage: 67.9% | disk: ↓ 0mb/s ↑ 0mb/s |
```

3. ngrok 설치(로컬 서버를 외부에서 접근 가능하게 해주는 기능)

(1) <https://ngrok.com/download/windows> 공식 사이트 접속 후 파일 다운로드

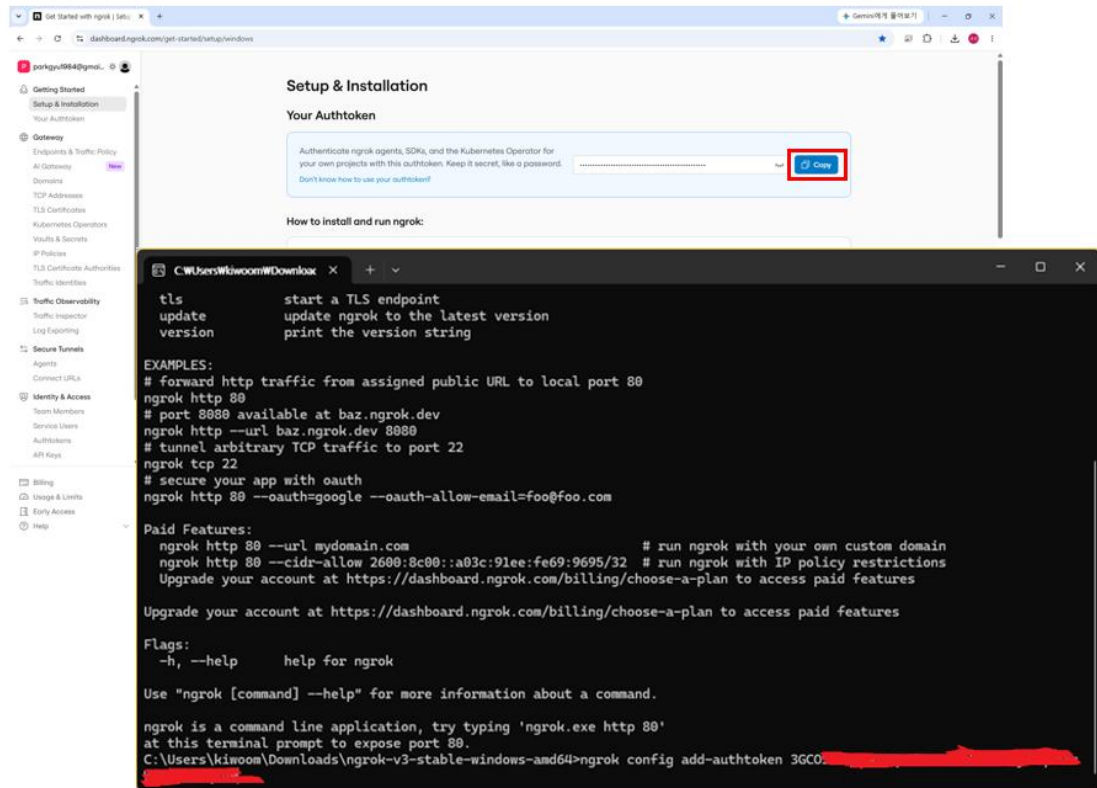


(2) 다운받은 파일 압축 해제 후 ngrok.exe 실행하여 ngrok터미널 오픈



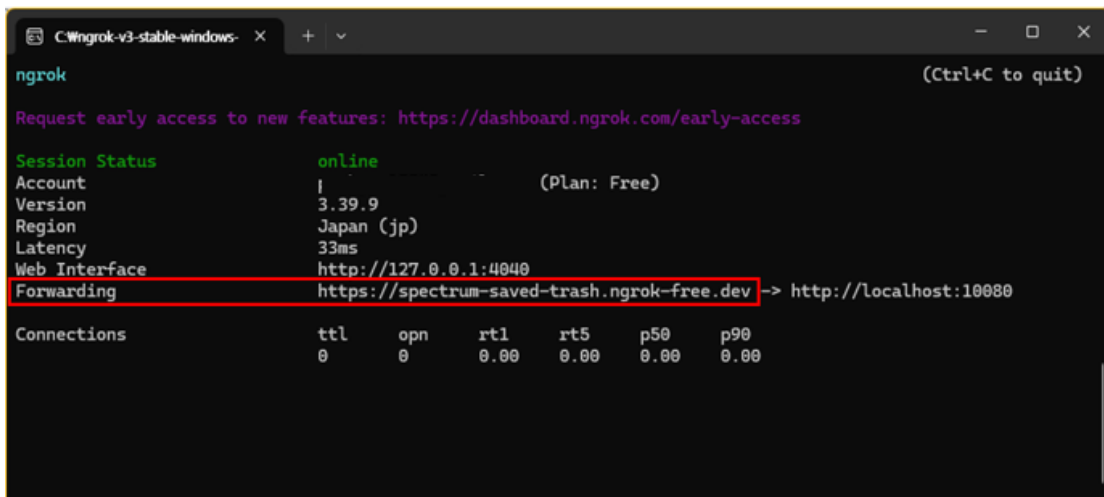
(3) ngrok 홈페이지 로그인하여 authtoken 복사 후 ngrok터미널에 아래 명령어 입력

ngrok config add-authtoken <복사한토큰붙여넣기>



ngrok http 10080 (.env의 PORT번호)

C:\ngrok-v3-stable-windows-amd64>ngrok http 10080

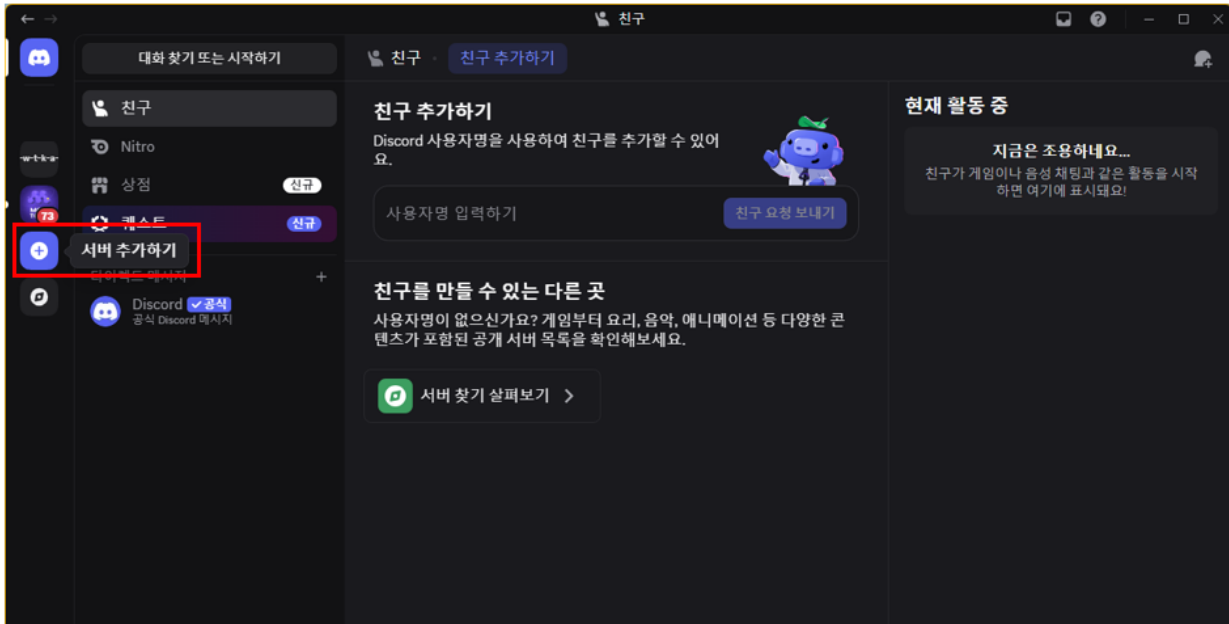


※ 위 화면에서 Forwarding 주소를 트레이딩 뷰에서 설정하게 됩니다. 창을 닫으면 동작하지 않으며 ngrok 재실행 마다 주소는 변경될 수 있습니다.

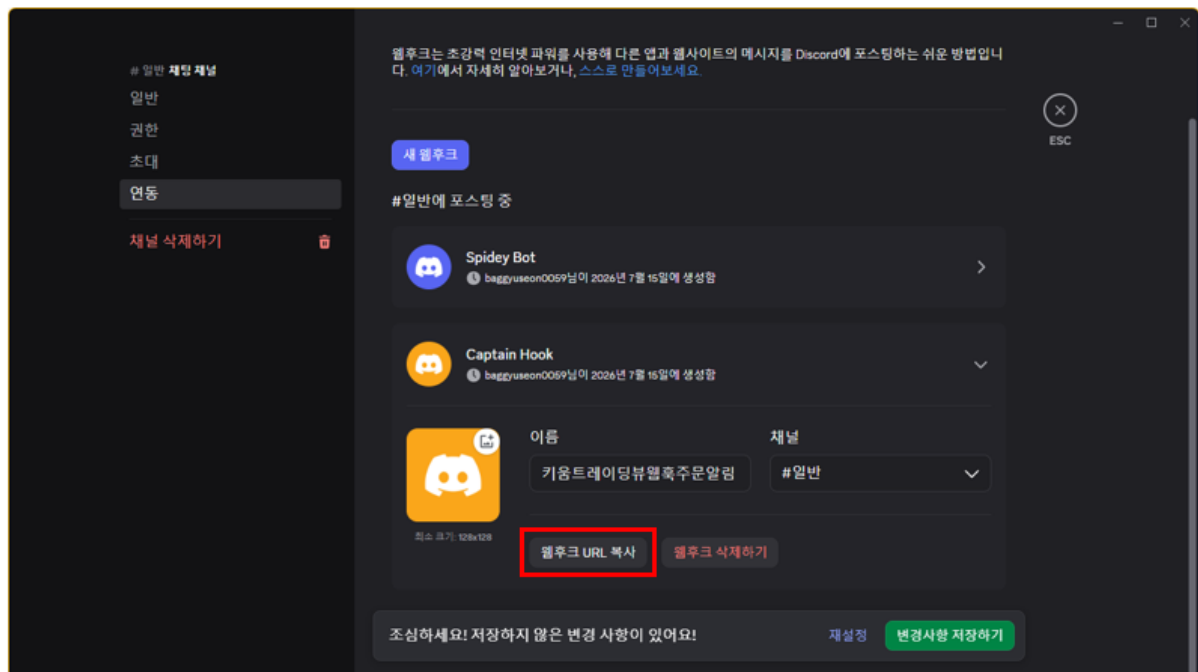
4. 트레이딩뷰 웹훅으로 메신저 알람 보내기

(1) 디스코드 앱에서 웹훅 추가

① 서버 추가하기 -> 추가한 서버 채널 편집 -> 연동 -> 웹훅 만들기 -> 새 웹훅



② 이름 설정하고 웹훅 URL 복사 클릭



③ 복사한 웹훅 URL 을 .env 파일의 DISCORD_WEBHOOK_URL 값으로 붙여넣고 로컬 서버 재시작

```
TRADINGVIEW-WEBHOOK-TO-K...  .env
> node_modules
> .env M
> .gitignore
> .api-caller.js
> .common.js
> .index.js M
> .package-lock.json
> .package.json
> .README.md
1 # 키움 REST API 모의투자
2 API_DOMAIN=https://mockapi.kiwoom.com
3 APP_KEY=I7X6
4 SECRET_KEY=Y
5 DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/1526757879032315915/Uf 0xFV8Rd1qt
6 PORT=80
7
8 # 키움 REST API 실전투자
9 API_DOMAIN=https://api.kiwoom.com
10 APP_KEY=
11 SECRET_KEY=
12 DISCORD_WEBHOOK_URL=
13 PORT=80
```

(2) 트레이딩뷰 웹훅 URL 및 메시지 설정

① 트레이딩뷰 웹훅 URL 에 아래 주소를 입력

- <ngrok Forwarding 주소>/kiwoom/webhook/discord/msg/send

예) <https://spectrum-saved-trash.ngrok-free.dev/kiwoom/webhook/discord/msg/send>



② 트레이딩뷰 웹훅 메시지 설정



③ 디스코드 알람 메시지 확인

일반

2026년 7월 16일

키움트레이딩뷰웹훅주문알림 2026-07-16 오전 9:51
 {"ord_no": "0048893", "dmst_stex_tp": "KRX", "return_code": 0, "return_msg": "모의투자 매도주문완료"}
 {"ord_no": "0048951", "dmst_stex_tp": "KRX", "return_code": 0, "return_msg": "모의투자 매도주문완료"}

2026년 7월 21일

키움트레이딩뷰웹훅주문알림 어제 오전 10:58
 {"ord_no": "0081617", "dmst_stex_tp": "KRX", "return_code": 0, "return_msg": "모의투자 매수주문완료"}
 {"ord_no": "0081690", "dmst_stex_tp": "KRX", "return_code": 0, "return_msg": "모의투자 매도주문완료"}

2026년 7월 22일

키움트레이딩뷰웹훅주문알림 오후 3:03
 [{"dmstStexTp": "KRX", "strCd": "005930", "ordUv": "263000", "ordQty": "1", "trdeTp": "O", "condUv": ""},
 {"dmstStexTp": "KRX", "strCd": "005930", "ordUv": "261750", "ordQty": "1", "trdeTp": "O", "condUv": ""},
 {"strCd": "005930", "ordUv": "261500", "msg": "디스코드 메시지 테스트"},
 {"strCd": "005930", "ordUv": "262750", "msg": "디스코드 메시지 테스트"},
 {"strCd": "005930", "ordUv": "263000", "msg": "디스코드 메시지 테스트"},
 {"strCd": "005930", "ordUv": "263000", "msg": "디스코드 메시지 테스트"},
 {"strCd": "005930", "ordUv": "263000", "msg": "디스코드 메시지 테스트"},
 {"strCd": "005930", "ordUv": "262750", "msg": "디스코드 메시지 테스트: 005930종목코드 261750원 하락 돌파"}]

키움트레이딩뷰웹훅주문알림 오후 3:25
 "디스코드 메시지 테스트: 005930종목코드 261750원 하락 돌파"

5. 트레이딩뷰 웹훅으로 주식 매매 하기

(1) 트레이딩 뷰 웹훅 URL 에 아래에서 원하는 기능의 URL 을 입력

- ① <ngrok Forwarding 주소>/kiwoom/webhook/kr/buy : 국내주식 매수
- ② <ngrok Forwarding 주소>/kiwoom/webhook/kr/sell : 국내주식 매도
- ③ <ngrok Forwarding 주소>/kiwoom/webhook/us/buy : 미국주식 매수
- ④ <ngrok Forwarding 주소>/kiwoom/webhook/us/sell : 미국주식 매도

※ 국내주식 매수 URL 샘플 예시

- <https://spectrum-saved-trash.ngrok-free.dev/kiwoom/webhook/kr/buy>



(2) 트레이딩 뷰 메시지 설정

① 국내 매수/매도 메시지 형식 및 설명

```
{
  "dmstStexTp": "KRX",
  "strCd": "{{ticker}}",
  "ordUv": "{{close}}",
  "ordQty": "1",
  "trdeTp": "0",
  "condUv": ""
}
```

dmstStexTp : 국내거래소구분코드, KRX, NXT, SOR(모의투자자는 KRX 만 가능)

- strCd : 종목코드
- ordUv : 매수/매도 가격
- ordQty : 매수/매도 수량
- trdeTp : 0:보통, 3:시장가, 5:조건부지정가, 81:장마감후시간외, 61:장시작전시간외, 62:시간외단일가, 6:최유리지정가, 7:최우선지정가, 10:보통(IOC), 13:시장가(IOC), 16:최유리(IOC), 20:보통(FOK), 23:시장가(FOK), 26:최유리(FOK), 28:스톱지정가, 29:중간가, 30:중간가(IOC), 31:중간가(FOK)
- condUv : 스톱지정가(28)인 경우 필수 입력



② 미국 매수/매도 메시지 형식 및 설명

```
{
  "stexTp" : "NY",
  "strCd" : "{{ticker}}",
  "ordUv" : "{{close}}",
  "ordQty" : "1",
  "trdeTp" : "00",
  "stop_pric" : ""
}
```

stexTp : 미국거래소코드구분코드, NA: AMEX, ND: NASDAQ, NY: NYSE

- strCd : 종족코드

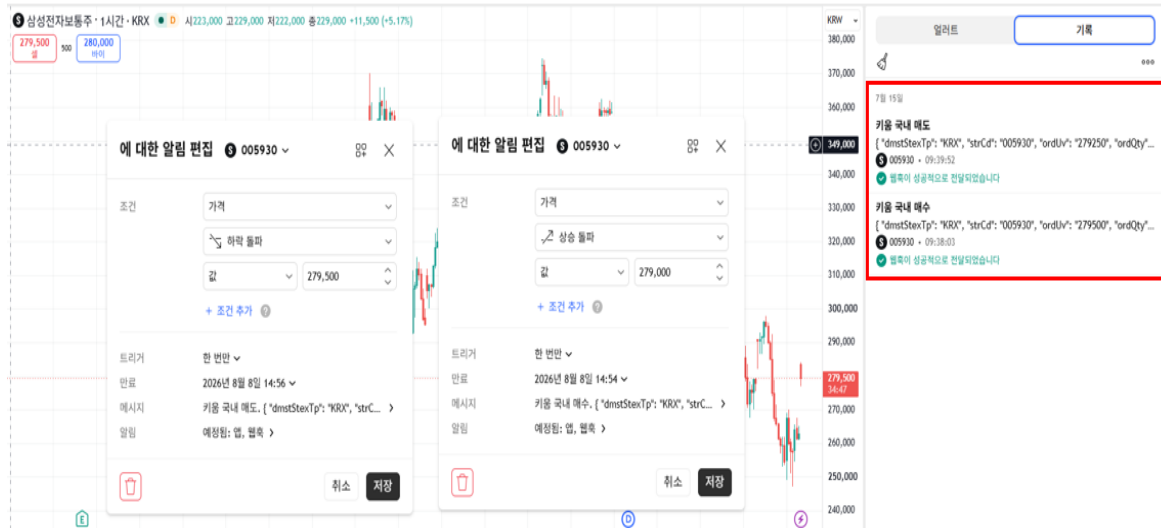
- ordUv : 매수/매도 가격
- ordQty : 매수/매도 수량
- trdeTp : 00:지정가, 03:시장가, 26:VWAP 지정가, 27:TWAP 지정가, 30:LOC, 36:VWAP 시장가, 37:TWAP 시장가
- stopPric : STOP LIMIT(34), STOP(35)인 경우 필수 입력



6. 트레이딩뷰 웹훅 -> 키움 REST API 주문 확인

※ 실제 주문이 발생합니다. 모의투자로 충분히 테스트 후 실전 투자로 전환해주세요.

(1) 트레이딩뷰 사이트에서 웹훅 전달 성공 확인



(2) 키움 트레이딩뷰 웹훅 서버 로그에서 키움 REST API 매수/매도 성공 확인

```
kiwoom@kiwoom-:~$ npm run local
$ npm run local
> tradingview-webhook-to-kiwoom-api-server@1.0.0 local
> node index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injected env (5) from .env // tip: enable debugging { debug: true }
2026-07-15 09:36:03 => start! tradingview-webhook-to-kiwoom-api server on port 80
2026-07-15 09:38:01 => > 트레이딩뷰 웹훅 - 국내 주식 매수 실행
2026-07-15 09:38:01 => - 요청: {"dmst_stex_tp":"KRX","stk_cd":"005930","ord_qty":"1","ord_uv":"279500","trde_tp":"0","cond_uv":""}
2026-07-15 09:38:01 => - 응답: {"ord_no":"0042136","dmst_stex_tp":"KRX","return_code":0,"return_msg":"모의투자 매수주문완료"}
2026-07-15 09:39:49 => > 트레이딩뷰 웹훅 - 국내 주식 매도 실행
2026-07-15 09:39:49 => - 요청: {"dmst_stex_tp":"KRX","stk_cd":"005930","ord_qty":"1","ord_uv":"279500","trde_tp":"0","cond_uv":""}
2026-07-15 09:39:49 => - 응답: {"ord_no":"0043339","dmst_stex_tp":"KRX","return_code":0,"return_msg":"모의투자 매도주문완료"}
```

(3) 매수/매도 완료 시 알람 메시지 확인

